

# Geometric redundancy resolution in inverse kinematics

Marko Guberina

March 31, 2026

## 1 Introduction

The concept of manipulability has been around since 1985. The idea is very straightforward. Take a ball of joint velocities in joint space, and map it, via the Jacobian, to operational space. The output is an ellipsoid. This ellipsoid tells you how easy it is to move in a particular direction, or, equivalently, how sensitive you are to forces in a particular direction. For the most part, it was used only to judge the quality of a particular posture. If you have a 6dof robot, there is no need to do anything else, as you can simply select the IK solution with the highest score. But, if you have a redundant robot, there are whole dofs to explore. However, in the vast majority of cases, the focus was on the volume of the ellipsoid, or on the length of the shortest axis (as at singularity the shortest axis of the ellipsoid is 0 in length) [TODO: have some citations about the pre-geometric use of MEs in redundant robots]. There were some minor efforts in designing a target manipulability ellipsoid (ME) [Lee89; LO16], but nothing came of that as far as I'm aware, as computing and setting up task compatibility is extremely time-consuming and tedious.

A turning point came with [Jaq+21] when the authors finally introduce differential geometry to manipulability ellipsoids. We've known about screw theory and the benefits of interpreting rigid body transformations as Lie group elements since at least the 90s, but noone bothered to extend the geometric approach to manipulability before. With this we can actually track prescribed manipulability ellipsoids, instead of just computing them.

So far there have been only 2 (and a half) applications of manipulability tracking. The first is in learning from demonstrations, specifically from a human. Humans can do most tasks we care about, but they have a different kinematic structure than the target robot. While you can track the end-effector position, you can't do anything with just joint angles (as they don't map to the robot), and you lose all information about posture, which is critical when you need to exert high forces for example. But you can find the human's manipulability ellipsoid, and then try to achieve that as it is independent from the underlying kinematic structure! Well, not exactly, but there is a mapping between the

human’s ME and the robot’s ME [RFH22]. The second one is singularity avoidance, for example [Mar+21]. The third one isn’t even manipulability tracking, just manipulability index (ellipsoid volume) maximization.

Ideally, we don’t need demonstrations. Instead, we build an algorithm which can generate a solution for the given motion problem from first principles, and then execute the solution — all you had to do was specify the task. For example, there are many tasks in which we know which path we want the robot to follow, ex. welding, polishing etc.

Moreover, sometimes we just want to move the robot a bit. As it turns out redundancy resolution is absolutely critical when there are left-over dofs to exploit. In fact, the higher the nullspace dimension, the more important it is to do some kind of redundancy resolution.

Let’s look at this case a bit more carefully: there is a good prescribed path, and we want to track it. To keep things simple for now, let’s say there are no external contacts and forces, i.e. the path is in free-space. The existing redundancy resolution methods don’t really offer anything to directly benefit in this task. As noted in [Lee89] [TODO: or wherever idk now], manipulability maximization can actually hurt performance. Singularity avoidance helps you avoid singularities, but that’s it. Conceivably, one should be able to design a useful and feasible redundancy resolution method, which would then avoid singularities as a byproduct. Thus, it is sensible to try to design manipulability ellipsoids strictly from the path geometry, and use them for redundancy resolution.

This was done in [YSV21].

## 1.1 Why is redundancy resolution still useful?

**Isn’t this magically solved during optimization?** Here one could say, ah, but we have trajectory optimization, so what is the point of crafting these additional structures when you could just throw in all your costs and constraints into an optimal control formulation? Depending on how the OCP is constructed, that might be the case. However:

- in general these OCPs are nonconvex, so you’re not actually finding the optimal solution
- MPC with manipulability maximization has higher manipulability than MPC without it (TODO: prove by demonstration), so you can (and should) put it as a conditioning cost
- you can put this in your traj. opt. and get a better conditioned problem, allowing you to add more costs/constraints
- even if traj. opt. did somehow leverage manipulability, it would be good to actually show this - more understanding, more profit

The goal of is to leverage the manipulability tracking controllers described in [Jaq+18].

All the relevant mathematics can be found in the paper text, although a reader without experience with differential geometry will most likely not be able to parse the finer details. Hence this text will only concern itself with methodology.

## 2 Manipulability tracking as a QP cost

Throughout the text, we restrict ourselves to 3D linear velocities. The methodology is identical for the full 6D case including angular velocities, but the targets and effect on manipulator posture are more difficult to visualize and assess. Hence  $v \in \mathbb{R}^3$ ,  $J \in \mathbb{R}^{3 \times n}$  and so on. To cover both cases, we'll use  $d \in \{3, 6\}$ . Assume there is a target manipulability ellipsoid  $\Lambda \in S_{++}^D \subset \mathbb{R}^{3 \times 3}$ . The goal is to, for one instant in time, compute the vector of joint velocities such that the current manipulability ellipsoid  $M$ . This vector can then be used as the main kinematic task, or a secondary task. In the paper, the secondary task is achieved via direct nullspace projection. However, this library (and myself) favour posing kinematic tasks as quadratic programs (QPs) solved by appropriate QP solvers as this framework makes it easier to combine different tasks, and, more importantly, linear constraints. Thus the very minor contribution of this work, apart from the implementation leveraging Pinocchio, is the formulation of manipulability tracking as a quadratic cost.

### 2.1 Derivation of target tangent vector in SPD space

Utilizing the following inner product at  $T_M S_{++}^D$ :

$$\langle \Sigma_1, \Sigma_2 \rangle_M = \text{tr} \left( M^{-\frac{1}{2}} \Sigma_1 M^{-1} \Sigma_2 M^{-\frac{1}{2}} \right) \quad (1)$$

as the Riemmanian norm, we can from the exponential and logarithmic maps on the SPD manifold:

$$\Lambda = \exp_M L = M^{\frac{1}{2}} \exp(M^{-\frac{1}{2}} L M^{-\frac{1}{2}}) M^{\frac{1}{2}} \quad (2)$$

$$L = \log_M \Lambda = M^{\frac{1}{2}} \log(M^{-\frac{1}{2}} \Lambda M^{-\frac{1}{2}}) M^{\frac{1}{2}} \quad (3)$$

Thus, provided the current manipulability ellipsoid  $M$  and the target manipulability ellipsoid  $\Lambda$ , we get the tangent vector from  $M$  to  $\Lambda$  as  $L = \log_M(\Lambda)$ . Now we need to compute the vector of joint velocities  $v$  such that we follow  $L$ .

### 2.2 Computing the derivative of the manipulability ellipsoid w.r.t. joint velocities

We begin with the obvious computations:

$$M = J J^T \text{ definition of kinematic manipulability ellipsoid} \quad (4)$$

$$\dot{M} = \dot{J} J^T + J \dot{J}^T \text{ applying } \frac{d}{dt} \quad (5)$$

Here we set  $\dot{M} = L$  to get the change in the manipulability ellipsoid we want. To make this a QP cost, we need a quadratic of  $\dot{q}$ . To get that, we need  $\dot{M} = \mathcal{J} \times_3 \dot{q}^T$ . From this we can do:

$$\min_{\dot{q}} \left\| \dot{M} - \mathcal{J} \times_3 \dot{q}^T \right\| \quad (6)$$

$$= \min_{\dot{q}} \left\| \text{vec}(\dot{M}) - \mathcal{J}_{(3)}^T \dot{q} \right\| \quad (7)$$

This is just rearranged linear algebra and it's better not to think about it too much — if the result is the same, it does not matter in which order one multiplies and sums. But to make at least notation less confusing,  $\text{vec}(\cdot)$  is matrix vectorization (stack matrix rows into a single column vector (I think)), and  $\times_3$  is tensor matricization along the 3rd index (same idea as vectorization, the point is to write the tensor product as matrix vector multiplication and that's it). The point here is we need to get this  $\mathcal{J}$ . It can also be written like this:

$$\mathcal{J} = \frac{\partial J}{\partial q} \times_2 J + \frac{\partial J^T}{\partial q} \times_1 J \quad (8)$$

We need to look into  $\dot{J}$  a bit, from the perspective of differentiating the forward kinematics map:

$$T = f(q) \text{ forward kinematics} \quad (9)$$

$$\mathcal{V} = J\dot{q} \text{ inverse kinematics} \quad (10)$$

$$\mathcal{A} = \dot{J}\dot{q} + J\ddot{q} \quad (11)$$

This means  $\dot{J}$  has a relation to acceleration, which we can use in computation via Pinocchio.

$$\frac{dJ(q(t))}{dt} = \frac{\partial J}{\partial q} \dot{q} \quad (12)$$

$$= \left[ \frac{\partial J}{\partial q_0} \dot{q} \dots \frac{\partial J}{\partial q_n} \dot{q} \right] \quad (13)$$

where  $H = \frac{\partial J}{\partial q} \in \mathbb{R}^{d \times d \times n}$  is the forward kinematics Hessian. We can't conveniently compute this directly. However, Pinocchio offers the possibility to conveniently compute  $\dot{J}$ . Looking at the last expression, we can get  $H$  by setting  $\dot{q} = e_i$  for every joint  $i$ .

Finally, we have all the components in a classic quadratic cost. This yields the canonical quadratic cost  $x^T P x + q^T x$  as follows:

$$P = \mathcal{J}^T \mathcal{J} + \lambda I \quad (14)$$

$$q = -2 * \text{vec}(L)^T \mathcal{J} \quad (15)$$

where we also threw in Tikhonov damping as standard practise ( $\mathcal{J}$  needs to be inverted within the QP solver, same story as when you do a pseudoinverse.)

### 3 Important question

#### 3.1 How to know which manipulability ellipsoid to go to?

In the paper, manipulability ellipsoids are learned from demonstration. This is nifty because the demonstrator can have a different embodiment — the manipulability ellipsoid stays the same.

However, given a fully specified task such as door opening, one should conceivably be able to construct an appropriate manipulability ellipsoid. While it is obvious that, in the kinematic manipulability case, the ellipsoid should be elongated along the direction of motion, the quantitative question of how much remains unanswered. The same goes for singularity avoiding strategies.

Answering this question is future work for myself.

### References

- [Lee89] Sukhan Lee. “Dual redundant arm configuration optimization with task-oriented dual arm manipulability”. In: *IEEE Transactions on Robotics and Automation* 5.1 (1989), pp. 78–97.
- [LO16] Inho Lee and Jun-Ho Oh. “Humanoid posture selection for reaching motion and a cooperative balancing controller”. In: *Journal of Intelligent & Robotic Systems* 81.3 (2016), pp. 301–316.
- [Jaq+18] Noémie Jaquier et al. “Geometry-aware Tracking of Manipulability Ellipsoids.” In: *Robotics: Science and Systems*. CONF. 2018.
- [Jaq+21] Noémie Jaquier et al. “Geometry-aware manipulability learning, tracking, and transfer”. In: *The International Journal of Robotics Research* 40.2-3 (2021), pp. 624–650.
- [Mar+21] Filip Marić et al. “A Riemannian metric for geometry-aware singularity avoidance by articulated robots”. In: *Robotics and autonomous systems* 145 (2021), p. 103865.
- [YSV21] Lei Yan, Theodoros Stouraitis, and Sethu Vijayakumar. “Decentralized ability-aware adaptive control for multi-robot collaborative manipulation”. In: *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 2311–2318.
- [RFH22] Anna Reithmeir, Luis Figueredo, and Sami Haddadin. “Human-to-robot manipulability domain adaptation with parallel transport and manifold-aware ICP”. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2022, pp. 5218–5225.